



OFFICIAL DEVELOPER REFERENCE

ClickPesa PHP & Laravel SDK

Complete Integration Guide & API Reference Manual

UBITECH SOLUTIONS LIMITED

Professional Software Engineering & Digital Payment Integrations

 **Phone / Support: +255 766 192 332**

Repository: github.com/ubitecholutionsltd/clickpesa-laravel-package

Release Version: 1.0.1 • September 2026

1. Executive Summary & Architecture

The **ClickPesa PHP & Laravel SDK** is an enterprise-grade library engineered by **UBITECH SOLUTIONS LIMITED** to provide frictionless integration with ClickPesa's collection and disbursement APIs across Tanzania and East Africa.

Core Principles

- **Universal Compatibility (Any PHP Application):** Standalone core with zero framework lock-in. Compatible with PHP 8.1, 8.2, and 8.3 in vanilla PHP, Symfony, WordPress, or CLI tools.
- **First-Class Laravel Integration:** Out-of-the-box auto-discovery, publishable configuration, static `ClickPesa` Facade, native Cache token persistence, Webhook routing macro, and dispatched Laravel events.
- **Zero-Maintenance Authentication:** Automatic token management that handles issuance, in-memory/cache storage, and preemptive refresh before the 1-hour expiration limit.
- **Compliant Checksum Engine:** Built-in canonicalization and HMAC-SHA256 signing for outgoing requests and timing-safe webhook verification.
- **Built-in Testing Mock:** `ClickPesa::fake()` and assertion helpers for instant unit and integration testing without reaching production endpoints.

Note on ClickPesa Environment: ClickPesa operates directly against live production with small amounts and pre-KYC tier limits. There is no separate sandbox domain; all calls target <https://api.clickpesa.com/third-parties>.

2. Installation & Configuration

Installing with Composer

```
composer require ubitechsolutionsltd/clickpesa-laravel-package
```

Publishing Laravel Configuration

```
php artisan vendor:publish --tag=clickpesa-config
```

This generates `config/clickpesa.php`. Set your credentials in your `.env` file:

```
# .env
CLICKPESA_CLIENT_ID=your_client_id_here
CLICKPESA_API_KEY=your_api_key_here
CLICKPESA_CHECKSUM_KEY=your_checksum_secret_here
CLICKPESA_CHECKSUM_ENABLED=true
CLICKPESA_TIMEOUT=30
CLICKPESA_TOKEN_TTL_MARGIN=300
```

3. Authentication & Security Engine

Automatic Token Management

ClickPesa requires a JWT bearer token generated via `POST /generate-token` using the `client-id` and `api-key` request headers. The token is valid for exactly **1 hour**.

The SDK encapsulates this logic inside `ClickPesa\Auth\TokenManager`. On every authenticated request:

1. The SDK checks if a valid token exists in cache (in-memory for standalone PHP, or Laravel Cache in Laravel).
2. If expired or missing, it automatically calls ClickPesa's `/generate-token` API and stores the token with a 5-minute safety buffer.
3. If an unexpected 401 response occurs, it proactively flushes the token and transparently retries the call once.

Canonicalization & Checksum Engine

When checksum is enabled, ClickPesa enforces an HMAC-SHA256 signature calculated on a canonicalized JSON payload:

1. **Alphabetical Key Sorting:** Recursively sorts all associative array/object keys alphabetically at every level.
2. **Preserving Sequential Arrays:** Numeric lists maintain their original order while sorting nested objects.
3. **Excluding Checksum Fields:** Automatically strips `checksum` and `checksumMethod` fields before calculation.
4. **Compact JSON Serialization:** Serialized using `json_encode($data, JSON_UNESCAPED_SLASHES)`.
5. **HMAC-SHA256:** Signed using your secret checksum key.

```
use ClickPesa\Security\Checksum;

// Generate outgoing checksum
$hash = Checksum::generate($secretKey, $payload);

// Timing-safe verification for incoming webhooks
$isValid = Checksum::verify($secretKey, $payload, $receivedChecksum);
```

4. Payment Collections API

1. Mobile USSD-PUSH

Collect payments instantly by pushing USSD authorization prompts to customer handsets on Vodacom M-Pesa, Tigo-Pesa, Airtel Money, or Halopesa.

Preview Fees & Available Networks

```
use ClickPesa\Laravel\Facades\ClickPesa;

$preview = ClickPesa::ussdPush()->preview([
    'amount'          => '10000',
    'currency'         => 'TZS',
    'orderReference'  => 'ORD-1001',
    'phoneNumber'     => '255712345678',
    'fetchSenderDetails' => true,
]);
```

Initiate USSD-PUSH Prompt

```
$response = ClickPesa::ussdPush()->initiate([
    'amount'          => '10000',
    'currency'         => 'TZS',
    'orderReference'  => 'ORD-1001',
    'phoneNumber'     => '255712345678',
]);

// Returns:
// [
//   'id' => 'PUSH12345',
//   'status' => 'PROCESSING',
//   'channel' => 'TIGO-PESA',
//   'orderReference' => 'ORD-1001',
//   'collectedAmount' => '10000'
// ]
```

2. Card Payments

Accept payments from Visa, MasterCard, UnionPay, and American Express.

```

$card = ClickPesa::cardPayments()->initiate([
    'amount'          => '50',
    'currency'        => 'USD',
    'orderReference' => 'CARD-ORD-902',
    'customer' => [
        'fullName'    => 'Jane Doe',
        'email'       => 'jane@example.com',
        'phoneNumber' => '255712345678',
    ],
]);

$redirectLink = $card['cardPaymentLink'];

```

3. Hosted Checkout Links

Generate secure, hosted checkout pages supporting itemized orders or fixed totals.

```

$link = ClickPesa::checkoutLinks()->generate([
    'totalPrice'      => '45000',
    'orderReference' => 'INV-2045',
    'orderCurrency'   => 'TZS',
    'customerName'    => 'Mathayo John',
    'customerEmail'   => 'mathayo@example.com',
    'customerPhone'   => '255712345678',
    'description'     => 'Payment for Software License',
]);

$hostedUrl = $link['checkoutLink'];

```

5. BillPay Control Numbers & Direct Debit

BillPay Control Numbers

Generate standard 14-digit Tanzanian control numbers for offline and online customer payments via banks and mobile money wallets.

Create Order Control Number (One-Time Invoice)

```
$orderBill = ClickPesa::billPay()->createOrder([
    'billDescription' => 'Water Bill - September 2026',
    'billAmount'      => 90000,
    'billPaymentMode' => 'ALLOW_PARTIAL_AND_OVER_PAYMENT',
    'billReference'   => 'INV-SEPT-01',
]);
$controlNumber = $orderBill['billPayNumber'];
```

Create Customer Control Number (Reusable Account)

```
$custBill = ClickPesa::billPay()->createCustomer();
$customerControlNo = $custBill['billPayNumber'];
```

Bulk Control Numbers (Up to 50 Items)

```
$bulk = ClickPesa::billPay()->bulkCreateOrders([
    ['billDescription' => 'Invoice 101', 'billAmount' => 15000],
    ['billDescription' => 'Invoice 102', 'billAmount' => 25000],
]);
```

CRDB Direct Debit Mandates

Request recurring direct debit authorizations from customers' CRDB Bank accounts.

```
// Request Mandate Approval
$mandate = ClickPesa::crdbDirectDebit()->requestMandate([
    'accountNumber' => '0150123456700',
    'amount'        => 50000,
    'orderReference'=> 'MANDATE-01',
    'startDate'     => '2026-10-01',
    'frequency'     => 'MONTHLY',
]);

// Inspect Mandate Details
$details = ClickPesa::crdbDirectDebit()->getMandate('MANDATE_REQ_ID_123');

// Cancel Active Mandate
ClickPesa::crdbDirectDebit()->cancelMandate(['mandateRequestId' => 'MANDATE_REQ_ID_123']);
```

Querying Payments

```
// Query specific payment attempts by Order Reference
$attempts = ClickPesa::payments()->get('ORD-1001');

// Query with pagination and filtering
$allPayments = ClickPesa::payments()->all([
    'status'      => 'SUCCESS',
    'startDate'   => '2026-09-01',
    'limit'       => 25,
]);
```

6. Disbursements & Payouts API (All Types)

60-Second Cooldown: ClickPesa enforces a strict 60-second cooldown between payout creation requests per merchant. The SDK automatically catches this and raises a `RateLimitException` containing `$e->getRetryAfterSeconds()`.

1. Mobile Money (MNO) Payouts

Disburse funds directly to Vodacom M-Pesa, Airtel Money, Mixx by Yas (Tigo), or HaloPesa wallets.

```
// 1. Preview Fee & Available Balance
$preview = ClickPesa::mobileMoneyPayouts()->preview([
    'amount'      => 25000,
    'currency'     => 'TZS',
    'orderReference' => 'PAY-MNO-01',
    'phoneNumber'  => '255712345678',
]);
$fee = $preview['fee']; // e.g., 700 TZS

// 2. Execute Payout
$payout = ClickPesa::mobileMoneyPayouts()->create([
    'amount'      => 25000,
    'currency'     => 'TZS',
    'orderReference' => 'PAY-MNO-01',
    'phoneNumber'  => '255712345678',
]);
```

2. Bank Transfers (EFT & TISS)

Disburse payouts to any commercial bank account in Tanzania using Account Number and BIC. Transfers up to 20M TZS use EFT (fee: 2,360 TZS); transfers above 20M TZS must use TISS (fee: 11,800 TZS).

```
// 1. Preview Bank Payout
$preview = ClickPesa::bankPayouts()->preview([
    'amount'      => 500000,
    'currency'     => 'TZS',
    'orderReference'=> 'PAY-BNK-01',
    'accountNumber'=> '0150123456700',
    'bic'          => 'CORUTZTZ', // CRDB Bank BIC
]);

// 2. Execute Bank Payout
$bankPayout = ClickPesa::bankPayouts()->create([
    'amount'      => 500000,
    'currency'     => 'TZS',
    'orderReference'=> 'PAY-BNK-01',
    'accountNumber'=> '0150123456700',
    'accountName'  => 'Acme Trading Ltd',
    'bic'          => 'CORUTZTZ',
    'transferType' => 'ACH', // 'ACH' for EFT, 'RTGS' for TISS
]);
```

3. Merchant Lipa Namba Payouts

Send money directly to merchant Lipa Namba numbers via mobile networks.

```
// 1. Get providers (e.g. Vodacom 503, Airtel 502)
$providers = ClickPesa::lipaNambaPayouts()->providers();

// 2. Send to Lipa Namba
$lnPayout = ClickPesa::lipaNambaPayouts()->create([
    'amount'      => 15000,
    'currency'     => 'TZS',
    'orderReference'=> 'PAY-LN-01',
    'lipaNamba'    => '48001268',
    'providerCode' => '503',
]);
```

4. TIPS TanQR Payouts (Direct to QR Code)

```
$qrPayout = ClickPesa::lipaNambaPayouts()->create([
    'amount'      => 30000,
    'currency'     => 'TZS',
    'orderReference'=> 'PAY-QR-01',
    'qrCode'       => '00020101021226...',
]);
```

5. Hosted Payout Links

```
// Generate a link where payee selects mobile money or bank
$link = ClickPesa::payoutLinks()->create([
    'amount'      => 100000,
    'currency'    => 'TZS',
    'orderReference' => 'PAY-LINK-01',
    'recipientName' => 'Juma Hamisi',
]);
$claimUrl = $link['payoutUrl'];
```

6. Querying Payout Status & Pagination

```
// Query specific payout by Order Reference
$payoutStatus = ClickPesa::payouts()->get('PAY-MNO-01');

// Filter payout history with pagination
$history = ClickPesa::payouts()->all([
    'status'      => 'SUCCESS',
    'startDate'   => '2026-09-01',
    'limit'       => 50,
]);
```

7. Pricing, Fee Bearers & Offline Fee Calculator

ClickPesa differentiates between channels where fees are charged to the **customer** versus channels where fees are deducted from the **merchant**.

ClickPesa Tariff & Fee Bearers Matrix

Payment Channel	Fee Structure	Fee Bearer	Transaction Limits
USSD Push	Tiered Slab (54 – 7,960 TZS)	Customer	500 to 3,000,000 TZS
Card Payments	4.85%	Customer	Visa, Mastercard, UnionPay
BillPay (M-Pesa & Airtel)	1.0%	Merchant	500 to 5,000,000 TZS
BillPay (HaloPesa)	2.0%	Merchant	500 to 5,000,000 TZS
BillPay (Mixx by Yas / Tigo)	2.5%	Merchant	500 to 5,000,000 TZS
CRDB BillPay	1.0%	Merchant	1,000 to 100,000,000 TZS
CRDB Direct Debit	2,000 TZS Flat	Merchant	Up to 100,000,000 TZS
TIPS TanQR Collection	2.0%	Merchant	TIPS QR Merchant Code
Mobile Money / TanQR Payouts	Tiered Slab (52 – 9,890 TZS)	Configurable	100 to 5,000,000 TZS
Bank EFT Payout	2,360 TZS Flat	Configurable	Up to 20,000,000 TZS
Bank TISS Payout (TZS)	11,800 TZS Flat	Configurable	Up to 1,000,000,000 TZS
Bank TISS Payout (USD)	\$7.50 USD Flat	Configurable	Up to \$1,000,000 USD

Instant Offline Fee Estimation (`ClickPesa::fees()`)

```
// 1. Calculate USSD Push fee
$ussdFee = ClickPesa::fees()->calculateUssdPushFee(15000); // 920.0 TZS

// 2. Calculate Card Payment fee
$cardFee = ClickPesa::fees()->calculateCardFee(50000); // 2,425.0 TZS (4.85%)

// 3. Calculate BillPay fee
$mpesaFee = ClickPesa::fees()->calculateBillPayFee('mpesa', 100000); // 1,000.0 TZS (1%)

// 4. Calculate Net Merchant Settlement
$net = ClickPesa::fees()->calculateNetSettlement('billpay_mpesa', 100000); // 99,000.0 TZS

// 5. Payout Budgeting (Absorb Fee vs Deduct from Payee)
$absorbed = ClickPesa::fees()->calculatePayoutDeduction('mobile_money', 50000, absorbFee: true);
// Total merchant debited: 51,460 TZS; Payee receives: 50,000 TZS

$deducted = ClickPesa::fees()->calculatePayoutDeduction('mobile_money', 50000, absorbFee: false);
// Total merchant debited: 50,000 TZS; Payee receives: 48,540 TZS

// 6. Pre-Flight Limits Validation
ChannelLimits::validate('ussd_push', 15000); // Passes without network call!
```

Environment Variables for Tariff Configuration

All fees are fully customizable in `.env`:

```
# .env Pricing Overrides
CLICKPESA_FEE_CARD_PERCENT=4.85
CLICKPESA_FEE_BILLPAY_MPESA_PERCENT=1.0
CLICKPESA_FEE_BILLPAY_HALOPESA_PERCENT=2.0
CLICKPESA_FEE_BILLPAY_MIXX_PERCENT=2.5
CLICKPESA_FEE_BILLPAY_CRDB_PERCENT=1.0
CLICKPESA_FEE_CRDB_DIRECT_DEBIT=2000
CLICKPESA_FEE_BANK_EFT=2360
CLICKPESA_FEE_BANK_TISS_TZS=11800
CLICKPESA_FEE_BANK_TISS_USD=7.50
CLICKPESA_BANK_EFT_MAX_THRESHOLD=20000000
```

8. Webhooks & Event Processing

ClickPesa sends asynchronous HTTP POST notifications when transactions complete, fail, or reverse.

Setting Up Webhook Route in Laravel

Register the route macro in `routes/api.php` :

```
use Illuminate\Support\Facades\Route;

// Registers POST /clickpesa/webhooks
Route::clickpesaWebhooks('clickpesa/webhooks');
```

Listening to Dispatched Laravel Events

ClickPesa Event Name	Dispatched Laravel Event	Key Attributes
PAYMENT RECEIVED	ClickPesa\Laravel\Events\PaymentReceived	getOrderReference() , getCollectedAmount() , getCustomer()
PAYMENT FAILED	ClickPesa\Laravel\Events\PaymentFailed	getOrderReference() , getMessage()
PAYOUT INITIATED	ClickPesa\Laravel\Events\PayoutInitiated	getOrderReference() , getAmount() , getFee()
PAYOUT REFUNDED	ClickPesa\Laravel\Events\PayoutRefunded	getOrderReference() , getRefundMessage()
PAYOUT REVERSED	ClickPesa\Laravel\Events\PayoutReversed	getOrderReference() , getReverseMessage()
DEPOSIT RECEIVED	ClickPesa\Laravel\Events\DepositReceived	getDepositAmount() , getPaymentReference()

Example Listener Implementation

```
namespace App\Listeners;

use ClickPesa\Laravel\Events\PaymentReceived;

class MarkOrderAsPaid
{
    public function handle(PaymentReceived $event): void
    {
        $orderReference = $event->getOrderReference();
        $amount = $event->getCollectedAmount();
        $paymentRef = $event->getPaymentReference();

        // Update database order status
        $order = Order::where('reference', $orderReference)->first();
        if ($order) {
            $order->update([
                'status' => 'paid',
                'clickpesa_payment_reference' => $paymentRef,
            ]);
        }
    }
}
```

9. Standalone PHP & Application Testing

Using the SDK in Vanilla PHP / Non-Laravel Projects

```
require_once __DIR__ . '/vendor/autoload.php';

use ClickPesa\ClickPesaClient;

$client = new ClickPesaClient([
    'client_id'      => 'YOUR_CLIENT_ID',
    'api_key'        => 'YOUR_API_KEY',
    'checksum_key'   => 'YOUR_CHECKSUM_KEY',
    'checksum_enabled' => true,
]);

// Initiate USSD-PUSH
$res = $client->ussdPush()->initiate([
    'amount'        => '10000',
    'currency'       => 'TZS',
    'orderReference' => 'PHP_ORD_1',
    'phoneNumber'    => '255712345678',
]);
```

Testing & Mocking with `ClickPesa::fake()`

The SDK includes a first-class fake mechanism so your tests never make live network calls:

```

use ClickPesa\Laravel\Facades\ClickPesa;
use Tests\TestCase;

class CheckoutTest extends TestCase
{
    public function test_user_can_checkout(): void
    {
        // Mock all ClickPesa API calls
        ClickPesa::fake();

        $response = $this->postJson('/checkout', [
            'amount' => 5000,
            'phone' => '255712345678',
        ]);

        $response->assertOk();

        // Assert payment call occurred with expected parameters
        ClickPesa::assertInitiatedUssdPush(function ($data) {
            return $data['amount'] === '5000' && $data['phoneNumber'] === '255712345678';
        });

        // Assert no unauthorized payouts were triggered
        ClickPesa::assertNotSent('/payouts/create-bank-payout');
    }
}

```

10. Authors, Support & Commercial Inquiries



UBITECH SOLUTIONS LIMITED

We build custom payment gateways, enterprise fintech solutions, ERP integrations, and scalable mobile/web platforms across East Africa.

 **Telephone / WhatsApp: +255 766 192 332**

 **GitHub Organization:** github.com/ubitecholutionsltd

 **Package Repository:** [clickpesa-laravel-package](#)

Licensed under the MIT License. © 2026 UBITECH SOLUTIONS LIMITED. All rights reserved.